

Week 11 DAM Project

Ida, Frida & Mia

2026-12-03

```
knitr::opts_chunk$set(echo = TRUE)
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.2.0      ✓ readr      2.2.0
## ✓ forcats    1.0.1      ✓ stringr    1.6.0
## ✓ ggplot2     4.0.2      ✓ tibble     3.3.1
## ✓ lubridate  1.9.5      ✓ tidyr      1.3.2
## ✓ purrr       1.2.1
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to be
come errors
```

The task here is to load your Danish Monarchs csv into R using the `tidyverse` toolkit, calculate and explore the kings' duration of reign with pipes `%>%` in `dplyr` and plot it over time.

Load the kings

Make sure to first create an `.Rproj` workspace with a `data/` folder where you place either your own dataset or the provided `kings.csv` dataset.

1. Look at the dataset that are you loading and check what its columns are separated by? (hint: open it in plain text editor to see)

List what is the

separator: "reign_kings.csv" is seperated by "_"

2. Create a `kings` object in R with the different functions below and inspect the different outputs.

- `read.csv()`
- `read_csv()`
- `read.csv2()`
- `read_csv2()`

```
# FILL IN THE CODE BELOW and review the outputs
kings1 <- read.csv("data/kings.csv")

kings2 <- read_csv("data/kings.csv")
```

```
## Rows: 57 Columns: 11
## — Column specification —————
## Delimiter: ","
## chr (11): Name, House, Start_year, End_year, Birth_year, Death_year, Gender,...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
kings3 <- read.csv2("data/kings.csv")

kings4 <- read_csv2("data/kings.csv")
```

```
## i Using "','" as decimal and "'.'" as grouping mark. Use `read_delim()` for more control.
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 57 Columns: 1
## — Column specification —————
## Delimiter: ","
## chr (1): Name,House,Start_year,End_year,Birth_year,Death_year,Gender,Dynasty...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Answer: 1. Which of these functions is a tidyverse function? Read data with it below into a `kings` object # `read_csv` 2. What is the result of running `class()` on the `kings` object created with a tidyverse function. #`"spec_tbl_df"` `"tbl_df"` `"tbl"` `"data.frame"` 3. How many columns does the object have when created with these different functions? # 1 or 6 columns 4. Show the dataset so that we can see how R interprets each column

`kings`

```
# COMPLETE THE BLANKS BELOW WITH YOUR CODE, then turn the 'eval' flag in this chunk to TRUE.
```

```
kings <- read_csv("data/kings.csv", na = c("NULL",""))
```

```
## Rows: 57 Columns: 11
## — Column specification —————
## Delimiter: ","
## chr (7): Name, House, Gender, Dynasty, Source, BirthDMY, DeathDMY
## dbl (4): Start_year, End_year, Birth_year, Death_year
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
class(kings)
```

```
## [1] "spec_tbl_df" "tbl_df"      "tbl"        "data.frame"
```

```
glimpse(kings)
```

```
## Rows: 57
## Columns: 11
## $ Name      <chr> "Gorm den Gamle", "Harald 1. Blåtand", "Toke Gormsen", "Sve...
## $ House     <chr> "Gorm", "Gorm", "Gorm", "Gorm", "Gorm", "Gorm", "Gorm", "Fa...
## $ Start_year <dbl> 936, NA, 985, NA, 1014, 1018, 1035, 1042, 1047, 1074, 1080,...
## $ End_year   <dbl> 958, NA, 986, NA, 1018, 1035, 1042, 1047, 1074, 1080, 1086,...
## $ Birth_year <dbl> 908, 936, NA, NA, 963, NA, 995, 1018, 1024, 1019, 1041, 104...
## $ Death_year <dbl> 958, 985, 986, 1014, NA, 1035, 1042, 1047, 1076, 1080, 1086...
## $ Gender     <chr> "M", "M", "M", "M", "M", "M", "M", "M", "M", "M", "M", "M",...
## $ Dynasty    <chr> "Jellingdynastiet", "Jellingdynastiet", "Jellingdynastiet",...
## $ Source     <chr> "https://da.wikipedia.org/wiki/Gorm_den_Gamle", "https://da...
## $ BirthDMY   <chr> "908", "936", NA, "17/04/963", "994", "995", "1018", "1024"...
## $ DeathDMY   <chr> "958", "987", NA, "03/02/1014", "1018", "12/11/1035", "08/0...
```

```
head(kings)
```

```
## # A tibble: 6 × 11
##   Name      House Start_year End_year Birth_year Death_year Gender Dynasty Source
##   <chr>      <chr>      <dbl>   <dbl>      <dbl>      <dbl> <chr>   <chr>   <chr>
## 1 Gorm de... Gorm          936     958        908        958 M      Jellin... https...
## 2 Harald ... Gorm          NA      NA         936        985 M      Jellin... https...
## 3 Toke Go... Gorm          985     986         NA         986 M      Jellin... https...
## 4 Svend 1... Gorm          NA      NA         NA         1014 M      Jellin... https...
## 5 Harald ... Gorm         1014    1018        963         NA M      Jellin... https...
## 6 Knud 1... Gorm         1018    1035         NA         1035 M      Jellin... https...
## # i 2 more variables: BirthDMY <chr>, DeathDMY <chr>
```

Calculate the duration of reign for all the kings in your table

You can calculate the duration of reign in years with `mutate` function by subtracting the equivalents of your `startReign` from `endReign` columns and writing the result to a new column called `duration`. But first you need to check a few things:

- Is your data messy? Fix it before re-importing to R
- Do your start and end of reign columns contain NAs? Choose the right strategy to deal with them:
`na.omit()`, `na.rm=TRUE`, `!is.na()`

Create a new column called `duration` in the `kings` dataset, utilizing the `mutate()` function from `tidyverse`. Check with your group to brainstorm the options.

```
# YOUR CODE

kings %>%
  select(Start_year, End_year) %>%
  mutate(duration = End_year - Start_year)
```

```
## # A tibble: 57 × 3
##   Start_year End_year duration
##   <dbl>     <dbl>     <dbl>
## 1      936      958        22
## 2       NA       NA        NA
## 3      985      986         1
## 4       NA       NA        NA
## 5     1014     1018         4
## 6     1018     1035        17
## 7     1035     1042         7
## 8     1042     1047         5
## 9     1047     1074        27
## 10    1074     1080         6
## # i 47 more rows
```

Calculate the average duration of reign for all rulers

Do you remember how to calculate an average on a vector object? If not, review the last two lessons and remember that a column is basically a vector. So you need to subset your `kings` dataset to the `duration` column. If you subset it as a vector you can calculate average on it with `mean()` base-R function. If you subset it as a tibble, you can calculate average on it with `summarize()` tidyverse function. Try both ways!

- You first need to know how to select the relevant `duration` column. What are your options?
- Is your selected `duration` column a tibble or a vector? The `mean()` function can only be run on a vector. The `summarize()` function works on a tibble.
- Are you getting an error that there are characters in your column? Coerce your data to numbers with `as.numeric()`.
- Remember to handle NAs: `mean(X, na.rm=TRUE)`

```
kings %>%
  select(Start_year, End_year) %>%
  mutate(duration = End_year - Start_year) %>%
  summarise(duration = mean(duration, na.rm = TRUE))
```

```
## # A tibble: 1 × 1
##   duration
##   <dbl>
## 1    19.6
```

How many and which kings enjoyed a longer-than-average duration of reign?

You have calculated the average duration above. Use it now to `filter()` the `duration` column in `kings` dataset. Display the result and also count the resulting rows with `count()`

```
kings %>%
  select(Name, Start_year, End_year) %>%
  mutate(duration = End_year - Start_year) %>%
  filter(duration > mean(duration, na.rm = TRUE))
```

```
## # A tibble: 25 × 4
##   Name                Start_year End_year duration
##   <chr>                <dbl>   <dbl>   <dbl>
## 1 Gorm den Gamle         936     958     22
## 2 Svend 2. Estridsen    1047    1074     27
## 3 Niels                  1104    1134     30
## 4 Valdemar 1. den Store  1157    1182     25
## 5 Knud 4.                1182    1202     20
## 6 Valdemar 2. Sejr      1202    1241     39
## 7 Erik 5. Klipping      1259    1286     27
## 8 Erik 6. Menved        1286    1319     33
## 9 Valdemar 4. Atterdag   1340    1375     35
## 10 Erik 7. af Pommern    1396    1439     43
## # i 15 more rows
```

How many days did the three longest-ruling monarchs rule?

- Sort kings by reign `duration` in the descending order. Select the three longest-ruling monarchs with the `slice()` function
- Use `mutate()` to create `Days` column where you calculate the total number of days they ruled
- BONUS: consider the transition year (with 366 days) in your calculation!

```
kings %>%
  select(Name, Start_year, End_year) %>%
  mutate(duration = End_year - Start_year) %>%
  arrange(desc(duration)) %>%
  slice(1:3) %>%
  mutate(Days = duration * 365)
```

```
## # A tibble: 3 × 5
##   Name                Start_year End_year duration  Days
##   <chr>                <dbl>   <dbl>   <dbl> <dbl>
## 1 Christian 4.         1588    1648     60 21900
## 2 Margrete 2.         1972    2023     51 18615
## 3 Erik 7. af Pommern   1396    1439     43 15695
```

Challenge: Plot the kings' duration of reign through time

What is the long-term trend in the duration of reign among Danish monarchs? How does it relate to the historical violence trends ?

- Try to plot the duration of reign column in `ggplot` with `geom_point()` and `geom_smooth()`
- In order to peg the duration (which is between 1-99) somewhere to the x axis with individual centuries, I recommend creating a new column `midyear` by adding to `startYear` the product of `endYear` minus the `startYear` divided by two (`startYear + (endYear-startYear)/2`).
- Now you can plot the kings dataset, plotting `midyear` along the x axis and `duration` along y axis
- BONUS: add a title, nice axis labels to the plot and make the theme B&W and font bigger to make it nice and legible!

```
# YOUR CODE
```

And to submit this rmarkdown, knit it into html. But first, clean up the code chunks, adjust the date, rename the author and change the `eval=FALSE` flag to `eval=TRUE` so your script actually generates an output. Well done!

```
library(tidyverse) library(tidyverse)
```